C input and output

- The printf() and scanf() functions are used for input and output in C language.
- Both functions are inbuilt library functions, defined in stdio.h (header file).

C output

In C programming, printf() is one of the main output function. The function sends formatted output to the screen. For example,

Example 1: C Output

```c
#include <stdio.h>
int main()
{
    // Displays the string inside quotations
    printf("C Programming");
    return 0;
}
```

Run Code »

Output

```
C Programming
```

C output cont..

Example 2: Integer Output

```c
#include <stdio.h>
int main()
{
    int testInteger = 5;
    printf("Number = %d", testInteger);
    return 0;
}
```

Run Code »

Output

```
Number = 5
```

C output cont..

Example 3: float and double Output

```c
#include <stdio.h>
int main()
{
    float number1 = 13.5;
    double number2 = 12.4;

    printf("number1 = %f\n", number1);
    printf("number2 = %lf", number2);
    return 0;
}
```

Run Code »

Output

```
number1 = 13.500000
number2 = 12.400000
```

C output cont..

Example 4: Print Characters

```c
#include <stdio.h>
int main()
{
    char chr = 'a';
    printf("character = %c", chr);
    return 0;
}
```

Run Code »

Output

```
character = a
```

C Input

- In C programming, scanf() is one of the commonly used function to take input from the user.
- The scanf() function reads formatted input from the standard input such as keyboards.

C Input cont..

Example 5: Integer Input/Output

```c
#include <stdio.h>
int main()
{
    int testInteger;
    printf("Enter an integer: ");
    scanf("%d", &testInteger);
    printf("Number = %d",testInteger);
    return 0;
}
```

Run Code »

Output

```
Enter an integer: 4
Number = 4
```

- Here, we have used %d format specifier inside the scanf() function to take int input from the user.
- When the user enters an integer, it is stored in the testInteger variable.
- Notice, that we have used &testInteger inside scanf().
- It is because &testInteger gets the address of testInteger, and the value entered by the user is stored in that address.

C Input cont..

Example 6: Float and Double Input/Output

```c
#include <stdio.h>
int main()
{
    float num1;
    double num2;

    printf("Enter a number: ");
    scanf("%f", &num1);
    printf("Enter another number: ");
    scanf("%lf", &num2);

    printf("num1 = %f\n", num1);
    printf("num2 = %lf", num2);

    return 0;
}
```

Run Code »

Output

```
Enter a number: 12.523
Enter another number: 10.2
num1 = 12.523000
num2 = 10.200000
```

C Input cont..

Example 7: C Character I/O

```c
#include <stdio.h>
int main()
{
    char chr;
    printf("Enter a character: ");
    scanf("%c",&chr);
    printf("You entered %c.", chr);
    return 0;
}
```

Run Code »

Output

```
Enter a character: g
You entered g
```

- When a character is entered by the user in the above program, the character itself is not stored. Instead, an integer value (ASCII value) is stored.
- And when we display that value using %c text format, the entered character is displayed. If we use %d to display the character, it's ASCII value is printed.

C Input cont..

I/O Multiple Values

Here's how you can take multiple inputs from the user and display them.

```c
#include <stdio.h>
int main()
{
    int a;
    float b;

    printf("Enter integer and then a float: ");

    // Taking multiple inputs
    scanf("%d%f", &a, &b);

    printf("You entered %d and %f", a, b);
    return 0;
}
```

Run Code »

Output

```
Enter integer and then a float: -3
3.4
You entered -3 and 3.400000
```

Print an Integer

```c
#include <stdio.h>
int main() {
    int num = 10;  // Declare and initialize
    printf("The value of num is: %d\n", num);
    return 0;
}
```

Declare and Initialize a Character

```c
#include <stdio.h>
int main() {
    char ch = 'A';  // Declare and initialize a character
    printf("The character is: %c\n", ch);
    return 0;
}
```

Declare and Initialize a Float

```c
#include <stdio.h>
int main() {
    float pi = 3.14;  // Declare and initialize a float
    printf("Value of pi: %.2f\n", pi);
    return 0;
}
```

Declare and Initialize Variables

c

```c
#include <stdio.h>
int main() {
    int a = 10;
    float b = 5.5;
    char c = 'A';
    printf("Integer: %d, Float: %.2f, Character: %c\n", a, b, c);
    return 0;
}
```

Declare and Print Multiple Variables

```c
#include <stdio.h>
int main() {
    int a = 5, b = 10;  // Declare and initialize multiple variables
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Assign Initial Values and Modify Later

```c
#include <stdio.h>
int main() {
    int a = 5;
    a = 10;  // Change value of variable
    printf("Updated value of a: %d\n", a);
    return 0;
}
```

Assign a Constant Value

```c
#include <stdio.h>
int main() {
    const float pi = 3.14159;  // Declare a constant variable
    printf("Value of pi: %.5f\n", pi);
    return 0;
}
```

Input and Output of Variables

```c
#include <stdio.h>
int main() {
    int x;
    printf("Enter an integer: ");
    scanf("%d", &x);
    printf("You entered: %d\n", x);
    return 0;
}
```

Find the Size of Data Types

```c
#include <stdio.h>
int main() {
    printf("Size of int: %lu bytes\n", sizeof(int));
    printf("Size of float: %lu bytes\n", sizeof(float));
    printf("Size of double: %lu bytes\n", sizeof(double));
    printf("Size of char: %lu bytes\n", sizeof(char));
    return 0;
}
```

Operators

- An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations.
- For example: + is an operator to perform addition.
- C has a wide range of operators to perform various operations.

C operators can be classified into number of categories :

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increment and Decrement Operators
6. Conditional Operators
7. Bitwise Operators
8. Special Operators

Operators – Arithmetic Operators

- An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

Operator	Meaning of Operator
+	addition or unary plus
-	subtraction or unary minus
*	multiplication
/	division
%	remainder after division (modulo division)

Operators – Arithmetic Operators (Example)

Example 1: Arithmetic Operators

```c
// Working of arithmetic operators
#include <stdio.h>
int main()
{
    int a = 9,b = 4, c;

    c = a+b;
    printf("a+b = %d \n",c);
    c = a-b;
    printf("a-b = %d \n",c);
    c = a*b;
    printf("a*b = %d \n",c);
    c = a/b;
    printf("a/b = %d \n",c);
    c = a%b;
    printf("Remainder when a divided by b = %d \n",c);

    return 0;
}
```

Operators – Relational Operators

- A relational operator checks the relationship between two operands.
- If the relation is true, it returns 1; if the relation is false, it returns value 0.
- Relational operators are used in decision making and loops.

Operator	Meaning of Operator	Example
==	Equal to	`5 == 3` is evaluated to 0
>	Greater than	`5 > 3` is evaluated to 1
<	Less than	`5 < 3` is evaluated to 0
!=	Not equal to	`5 != 3` is evaluated to 1
>=	Greater than or equal to	`5 >= 3` is evaluated to 1
<=	Less than or equal to	`5 <= 3` is evaluated to 0

Operators – Relational Operators (Example)

Example 4: Relational Operators

```c
// Working of relational operators
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10;

    printf("%d == %d is %d \n", a, b, a == b);
    printf("%d == %d is %d \n", a, c, a == c);
    printf("%d > %d is %d \n", a, b, a > b);
    printf("%d > %d is %d \n", a, c, a > c);
    printf("%d < %d is %d \n", a, b, a < b);
    printf("%d < %d is %d \n", a, c, a < c);
    printf("%d != %d is %d \n", a, b, a != b);
    printf("%d != %d is %d \n", a, c, a != c);
    printf("%d >= %d is %d \n", a, b, a >= b);
    printf("%d >= %d is %d \n", a, c, a >= c);
    printf("%d <= %d is %d \n", a, b, a <= b);
    printf("%d <= %d is %d \n", a, c, a <= c);

    return 0;
}
```

Operators – Logical Operators

- An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false.
- Logical operators are commonly used in decision making in C programming.

Operator	Meaning	Example
&&	Logical AND. True only if all operands are true	If c = 5 and d = 2 then, expression `((c==5) && (d>5))` equals to 0.
||	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression `((c==5) || (d>5))` equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression `!(c==5)` equals to 0.

Thank You